Vedhhavarshine B P
reg no: 251501247

DESIGN AND ANALYSIS OF ALGORITHM

GREEDY METHOD

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

ANSWER:

```
#include <stdio.h>


int main(){
   int v;


   if(scanf("%d", &v) == 0) return 1;
```

```
    int denomination[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int count = 0;

    for(int i = 0; i < 9; i++){
        if(v == 0) break;
        count += v/denomination[i];
        v %= denomination[i];
    }

    printf("%d", count);

    return 0;
}
```

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

Vedhhavarshine B P
reg no: 251501247

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b){
   return *(int *)a - *(int *)b;
}

int main(){
   int n, m;
   int g[30000], s[30000];

   scanf("%d", &n);

   for (int i = 0; i < n; i++) scanf("%d", &g[i]);
```

```
    scanf("%d", &m);

    for (int i = 0; i < m; i++) scanf("%d", &s[i]);

    qsort(g, n, sizeof(int), compare);
    qsort(s, m, sizeof(int), compare);

    int i = 0;
    int j = 0;

    while (i < n && j < m){
        if (s[j] >= g[i]) i++;
        j++;
    }

    printf("%d\n", i);

    return 0;
}
```

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$.
But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.
Input Format

Vedhhavarshine B P
reg no: 251501247

First Line contains the number of burgers
Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

3
5 10 7

Sample Output
76

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

Answer:

```
#include<stdio.h>
#include<math.h>
int main(){
   int n;
   scanf("%d",&n);
   int arr[n];
   for(int i=0;i<n;i++){
```

```
        scanf("%d",&arr[i]);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] < arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    int s=0;
    for(int i=0;i<n;i++){
        s+=(pow(n,i))*arr[i];
    }
    printf("%d",s);
}
```

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b)
{
   return *(int *)a - *(int *)b;
}

int main(){
   int n;
   scanf("%d", &n);
   int arr[n];
   for(int i = 0; i < n; i++) scanf("%d", &arr[i]);

   qsort(arr, n, sizeof(int), compare);

   int total = 0;
   for(int i = 0; i <n ;i++){
      total += arr[i] * i;
   }

   printf("%d", total);
```

```
    return 0;
}
```

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b)
{
    return *(int *)a - *(int *)b;
}
```

Vedhhavarshine B P
reg no: 251501247

```
int compareDescending(const void *a, const void *b)
{
   return *(int *)b - *(int *)a;
}

int main(){
   int n;
   scanf("%d", &n);
   int arr1[n], arr2[n];
   for(int i = 0; i < n; i++) scanf("%d", &arr1[i]);
   for(int i = 0; i < n; i++) scanf("%d", &arr2[i]);

   qsort(arr1, n, sizeof(int), compare);
   qsort(arr2, n, sizeof(int), compareDescending);

   int total = 0;

   for(int i = 0; i < n; i++){
      total += arr1[i] * arr2[i];
   }

   printf("%d", total);

   return 0;
}
```